Invisible WORDs

⚠️ **Security Warning**: Work in an isolated environment (virtual machine or container). Never open unknown files on your primary machine.

📋 Challenge Description

Do you recognize this cyberpunk baddie? We don't either. AI art generators are all the rage nowadays, which makes it hard to get a reliable known cover image. But we know you'll figure it out. The suspect is believed to be trafficking in classics. That probably won't help crack the stego, but we hope it will give motivation to bring this criminal to justice!

💡 Hints

- Something doesn't quite add up with this image...
- How's the image quality?

🧰 Tools Used

- `file` – Determine file type
- `strings` – Extract printable character sequences
- `exiftool` – Read metadata
- `binwalk` – Detect embedded files
- `xxd` – View hexadecimal representation
- **Python** – Custom script for data extraction
- `7z` – Extract ZIP archives
- `grep` – Search for flag patterns

🔍 Complete Solution

Step 1: Download and Prepare

Download the file `output.bmp` from the challenge page.

The image shows a cyberpunk-style character. The hint suggests something is “off” about the image quality – this often indicates steganography or hidden data.

Step 2: Basic File Inspection

```
file output.bmp
```

Expected output:

```
output.bmp: PC bitmap, Windows 98/2000 and newer format, 960 x 540 x 32, cbSize
2073738, bits offset 138
```

The image is a **32-bit BMP** (Windows V5 format), 960×540 pixels.

Step 3: Search for Visible Text

```
strings output.bmp | grep -i "picoCTF"
```

Output: (no results)

The flag isn't directly visible.

Step 4: Inspect Metadata

```
exiftool output.bmp
```

Output:

```
ExifTool Version Number         : 13.50
File Name                       : output.bmp
...
File Type                       : BMP
BMP Version                     : Windows V5
Image Width                     : 960
Image Height                    : 540
Bit Depth                       : 32
Compression                     : Bitfields
Red Mask                        : 0x00007c00
Green Mask                      : 0x000003e0
Blue Mask                       : 0x0000001f
Alpha Mask                      : 0x00000000
...
```

Nothing suspicious in the metadata.

Step 5: Scan with Binwalk

```
binwalk output.bmp
```

Output: (nothing)

No embedded files detected by standard signature scanning.

Step 6: Hexdump Analysis

Let's examine the raw hexadecimal data:

```
xxd output.bmp | head
```

Output:

```
00000000: 424d 8aa4 1f00 0000 0000 8a00 0000 7c00  BM............|.
00000010: 0000 c003 0000 1c02 0000 0100 2000 0300  ............ ...
00000020: 0000 00a4 1f00 232e 0000 232e 0000 0000  ......#...#.....
00000030: 0000 0000 0000 007c 0000 e003 0000 1f00  .......|........
00000040: 0000 0000 0000 4247 5273 0000 0000 0000  ......BGRs......
00000050: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000060: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000070: 0000 0000 0000 0000 0000 0200 0000 0000  ................
00000080: 0000 0000 0000 0000 0000 3867 504b 9552  ..........8gPK.R
00000090: 0304 c618 1400 ce3d 0000 104a 0800 6f56  .......=...J..oV
```

> **Key finding**: At offset `0x88`, we see `50 4b` – the ASCII characters **"PK"**! This is the magic signature for a ZIP archive.

But wait – a valid ZIP archive should start with `50 4b 03 04`. Here we have `50 4b 95 52`. The bytes are **interleaved** or offset.

Step 7: Identify the Pattern

The challenge name **"Invisible WORDs"** is a hint. In computer architecture, a **word** is a fixed-size unit of data (often 2 bytes or 4 bytes). The image is 32-bit (4 bytes per pixel). The hint suggests we need to extract data in **word-sized chunks**.

Looking at the hexdump, every 4 bytes seems to contain 2 bytes of real data and 2 bytes of padding/color data. The "PK" signature is split across bytes.

Pattern discovered: Extract bytes at indices 0, 1, 4, 5, 8, 9, ... (take 2 bytes, skip 2 bytes).

Step 8: Extract Hidden ZIP with Python

```python
import struct

def extract_zip_from_bmp(input_file, output_file):
    with open(input_file, 'rb') as f:
        data = f.read()

    # The ZIP signature PK\x03\x04 is split in the BMP data.
    # Analysis shows that the data is interleaved.
    # We extract bytes at indices 0, 1, 4, 5, 8, 9...
    # This corresponds to taking 2 bytes, skipping 2 bytes.

    extracted_data = bytearray()
    for i in range(0, len(data), 4):
        if i + 2 <= len(data):
            extracted_data.extend(data[i:i+2])

    with open(output_file, 'wb') as f:
        f.write(extracted_data)

    print(f"Extracted ZIP data saved to {output_file}")

# Run the extraction
extract_zip_from_bmp('output.bmp', 'extracted.zip')
```

Output:

```
Extracted ZIP data saved to extracted.zip
```

Step 9: Verify the Extracted File

```
file extracted.zip
```

Output:

```
extracted.zip: data
```

It's not yet recognized as a valid ZIP. Let's examine its hexdump:

```
xxd extracted.zip | head
```

Output:

```
00000000: 424d 1f00 0000 0000 0000 0000 0000 2000  BM.......... .
00000010: 0000 1f00 0000 0000 0000 0000 0000 0000  ................
00000020: 0000 0000 5273 0000 0000 0000 0000 0000  ....Rs..........
00000030: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000040: 0000 0000 0000 504b 0304 1400 0000 0800  ......PK........
00000050: 6d13 7056 68b4 3b04 af95 0200 82d8 0600  m.pVh.;.........
00000060: 1c00 1c00 5a6e 4a68 626d 746c 626e 4e30  ....ZnJhbmtlbnN0
00000070: 5a57 6c75 4c58 526c 6333 5175 6448 6830  ZWluLXRlc3QudHh0
00000080: 5554 0900 038e 7e12 648e 7e12 6475 780b  UT....~.d.~.dux.
00000090: 0001 0400 0000 0004 0000 0000 8cfd cd8e  ................
```

Now at offset `0x44`, we see the proper ZIP signature: **`50 4b 03 04`**! The extracted data is now a valid ZIP archive starting at offset `0x44`.

Step 10: Scan with Binwalk

```
binwalk extracted.zip
```

Output:

```
DECIMAL       HEXADECIMAL     DESCRIPTION
--------------------------------------------------------------------------------
70            0x46            Zip archive data, at least v2.0 to extract, compressed
size: 169391, uncompressed size: 448642, name: ZnJhbmtlbnN0ZWluLXRlc3QudHh0
169645        0x296AD         End of Zip archive, footer length: 22
```

The archive contains a file with a Base64-encoded name: `ZnJhbmtlbnN0ZWluLXRlc3QudHh0`

Step 11: Extract the Archive

```
binwalk extracted.zip -Me --run-as=root
```

Navigate to the extracted directory:

```
cd _extracted.zip.extracted
```

```
ls
```

Output:

```
46.zip
```

Extract this zip:

```
7z x 46.zip
```

Output:

```
46.zip  ZnJhbmtlbnN0ZWluLXRlc3QudHh0
```

Step 12: Decode the Filename

The filename `ZnJhbmtlbnN0ZWluLXRlc3QudHh0` is Base64-encoded. Decode it:

```
echo "ZnJhbmtlbnN0ZWluLXRlc3QudHh0" | base64 -d
```

Output:

```
frankenstein-test.txt
```

Step 13: Examine the Text File

```
file ZnJhbmtlbnN0ZWluLXRlc3QudHh0
```

Output:

```
ZnJhbmtlbnN0ZWluLXRlc3QudHh0: Unicode text, UTF-8 (with BOM) text, with CRLF, LF line terminators
```

Search for the flag:

```
grep -r "picoCTF" ZnJhbmtlbnN0ZWluLXRlc3QudHh0
```

Output:

```
At that age I became acquainted with the celebrated picoCTF{<REDACTED>}
```

✅ Final Result

```
picoCTF{<REDACTED>}
```

> **Note**: The actual flag is redacted here. In the real challenge, following these steps will reveal the complete flag.

🧠 Key Concepts

Concept	Description
Word (Computer Architecture)	The natural unit of data used by a processor. This challenge uses 2-byte words.
BMP File Structure	32-bit BMPs store pixel data in 4-byte chunks (Blue, Green, Red, Alpha).
Data Interleaving	Hidden data can be spread across every nth byte, requiring extraction before analysis.
PKZip Signature	ZIP files start with `50 4B 03 04` (“PK” followed by two control characters).
Binwalk	Tool for detecting and extracting embedded files.
Base64 Encoding	Used to obfuscate filenames within the archive.

💡 Pro Tips

1. **"Word" is a hint** – In computing, a word is typically 2 or 4 bytes. The challenge name suggests extracting data in word-sized chunks.
2. **32-bit BMPs have 4 bytes per pixel** – The interleaving pattern (take 2 bytes, skip 2 bytes) is a common steganography technique.
3. **Look for "PK" in hexdumps** – `50 4B` is a dead giveaway for ZIP archives, even if the signature is split.
4. **The extracted data may need trimming** – The first few bytes of `extracted.zip` were still BMP header remnants; the actual ZIP started later.
5. **Base64 filenames are common** – Always decode them; they often reveal the original file name.
6. **Binwalk with `-Me` extracts recursively** – This saves time when dealing with nested archives.

🔗 Alternative Python One-Liner for Extraction

If you prefer a one-liner instead of a full script:

```
python
open('extracted.zip', 'wb').write(bytes([b for i, b in enumerate(open('output.bmp', 'rb').read()) if i % 4 < 2]))
```

This extracts bytes at indices 0,1,4,5,8,9,... – same as the script but more compact.

🏁 Conclusion

The image `output.bmp` contained a hidden ZIP archive where the data was **interleaved** across the pixel bytes. The pattern was to take the first 2 bytes of every 4-byte chunk (2 bytes of hidden data, 2 bytes of image data).

After extracting the hidden bytes, we obtained a corrupted ZIP file. However, within that file, a valid ZIP signature (`50 4B 03 04`) appeared at offset `0x44` . Using `binwalk` , we extracted a nested archive containing a Base64-encoded filename. Decoding that filename revealed `franklin-test.txt` , which contained the flag.

This challenge demonstrates:

- **Data interleaving** – Hiding data by distributing it across every nth byte

- **Steganography in BMPs** – Using image pixel data as a carrier for hidden files
- **Forensic extraction** – Recovering nested archives with `binwalk`
- **Encoding detection** – Recognizing and decoding Base64 filenames

Key takeaway: When an image looks “off” or has unusual quality, examine its raw hex data. Look for patterns like repeating headers (`PK`) or interleaved data. Extracting bytes in word-sized chunks may reveal hidden archives.